Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory work #3

Non-Preemptive (bare-metal) operating systems

Author:
Alexandru Cebotari
std. gr. FAF-233

Verified:
Alexei Martiniuc

Chişinău 2026

Theory Background

This report covers only **Part 1** of Laboratory Work 3: non-preemptive bare-metal implementation with three modular tasks, recurrence/offset scheduler, visual feedback via LEDs, and periodic STDIO reporting. Part 2 (FreeRTOS) and control questions are intentionally excluded.

The cooperative multitasking model is based on run-to-completion tasks executed sequentially. Its main advantages are deterministic behavior, low overhead, and simple synchronization for shared software data in a single-core environment. The design target is Arduino Uno (ATmega328P, 16 MHz), simulated with Wokwi and built through PlatformIO.

Project repository: `https://github.com/Tirppy/si-course-repo/tree/main/Lab3/code`

The Task

Implement an embedded application that:

1. Detects button press/release transitions and measures each valid press duration.
2. Classifies press duration as short (< 500 ms) or long (≥ 500 ms), signaling with green/red LEDs.
3. Updates statistics and triggers yellow LED blink sequence (5 for short, 10 for long).
4. Reports every 10 seconds through STDIO over UART: total, short, long, average duration, then resets stats.
5. Uses a modular non-preemptive scheduler with context table (function pointer, recurrence, offset).

Technical implementation

The implementation follows modular separation in `Code/include` and `Code/src`: scheduler, three task modules, drivers (button, LEDs, STDIO bridge), and shared state module.

The code was refactored to a lower-level style for the non-preemptive bare-metal requirement. High-level Arduino API calls were removed from application modules and replaced with direct memory-mapped register access and interrupt-based timing. A dedicated `button_bridge` module is used to provide debounced button state to task logic.

- Hardware registers are accessed through a local abstraction in `mcu_registers.h` (no AVR header dependency in project source files).
- GPIO control remains register-level (button on Port D, LEDs on Port B).
- UART setup remains low-level, while reporting is exposed through a `stdio` bridge and `printf`.
- 1 ms time base uses Timer0 compare interrupt, bound through explicit vector symbol `__vector_14`.
- Atomic millisecond reads are protected by explicit interrupt disable/restore of `SREG`.

```
TaskContext g_tasks[] = {
  {task1_buttonDetectAndMeasure, 20U, 0U, 0U, true, "Task1"},
  {task2_statisticsAndBlink,     20U, 5U, 0U, true, "Task2"},
  {task3_periodicReport,      10000U,10U, 0U, true, "Task3"},
};

void schedulerTick(uint32_t nowMs) {
  (void)nowMs;
  for (size_t i = 0; i < g_taskCount; ++i) {
    if (g_tasks[i].enabled && g_tasks[i].countdownMs > 0U)
      --g_tasks[i].countdownMs;
  }

  for (size_t i = 0; i < g_taskCount; ++i) {
    TaskContext& task = g_tasks[i];
    if (task.enabled && task.countdownMs == 0U) {
      task.countdownMs = task.periodMs;
      task.taskFunction();
      return; // one task per tick
    }
  }
}
```

Library and header usage comparison (current project vs example report)

Current project external headers (low-level focus):

- `<stdint.h>`, `<stddef.h>`, and `<stdio.h>`.

- All MCU register and interrupt interactions are defined locally in `mcu_registers.h`.
- Button debouncing is encapsulated in `button_bridge.h/.cpp`.

Headers used in example report full-code listings:

- Same low-level core headers are present: `<avr/io.h>`, `<avr/interrupt.h>`, `<stdio.h>`, `<stdint.h>`.
- Project modules in example include `timer.h`, `serial.h`, `button.h`, `leds.h`, `scheduler.h`, `task1.h`, `task2.h`, `task3.h`.

Key comparison conclusion: Both implementations are bare-metal and register-oriented at runtime. The example keeps explicit AVR/system headers, while the current project centralizes register addresses and interrupt helpers in local modules, and uses `stdio` for formatted serial reporting. Functionally, module pairs remain equivalent (`platform_time.h`/`stdio_bridge.h`/`button_bridge.h` vs. `timer.h`/`serial.h`/direct button filtering).

Architecture and behavioral diagrams generated and included:

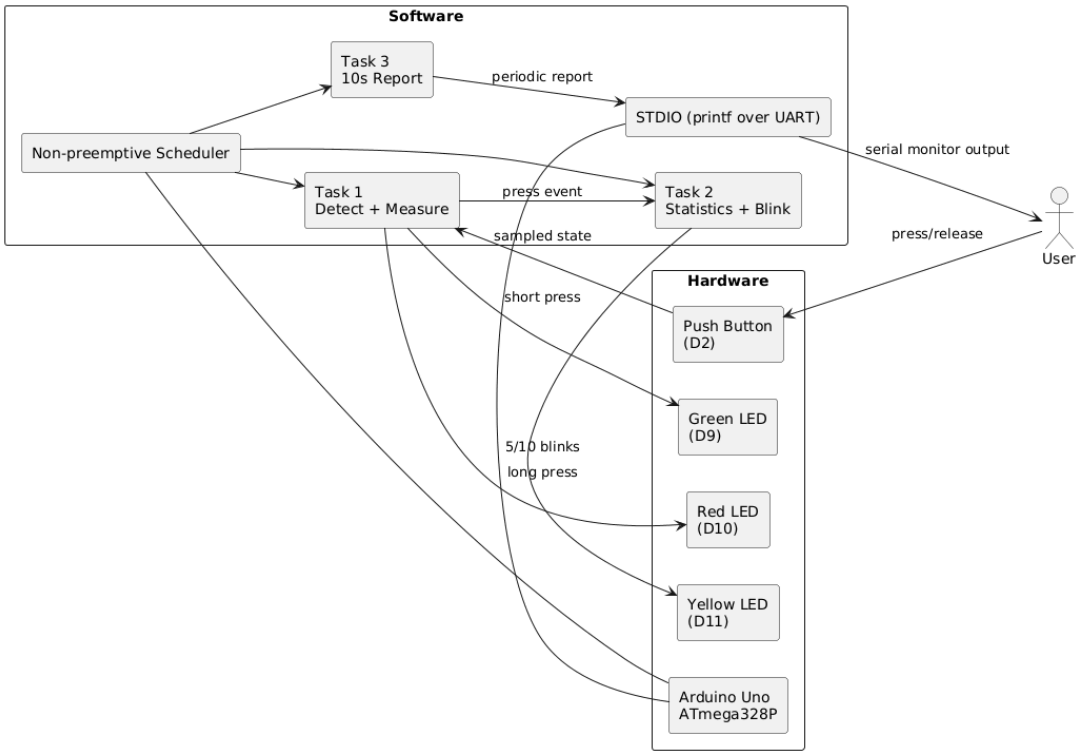


Figure 1: System structure diagram

This diagram presents the high-level architecture of the application, showing how button input is processed by the software stack and how output is delivered to LEDs and serial reporting. It identifies the main hardware elements (button and three LEDs),

the scheduler core, and the three service tasks that implement the required behavior. It also clarifies the direction of control and data propagation from user action to output channels, helping justify module boundaries and interaction order.

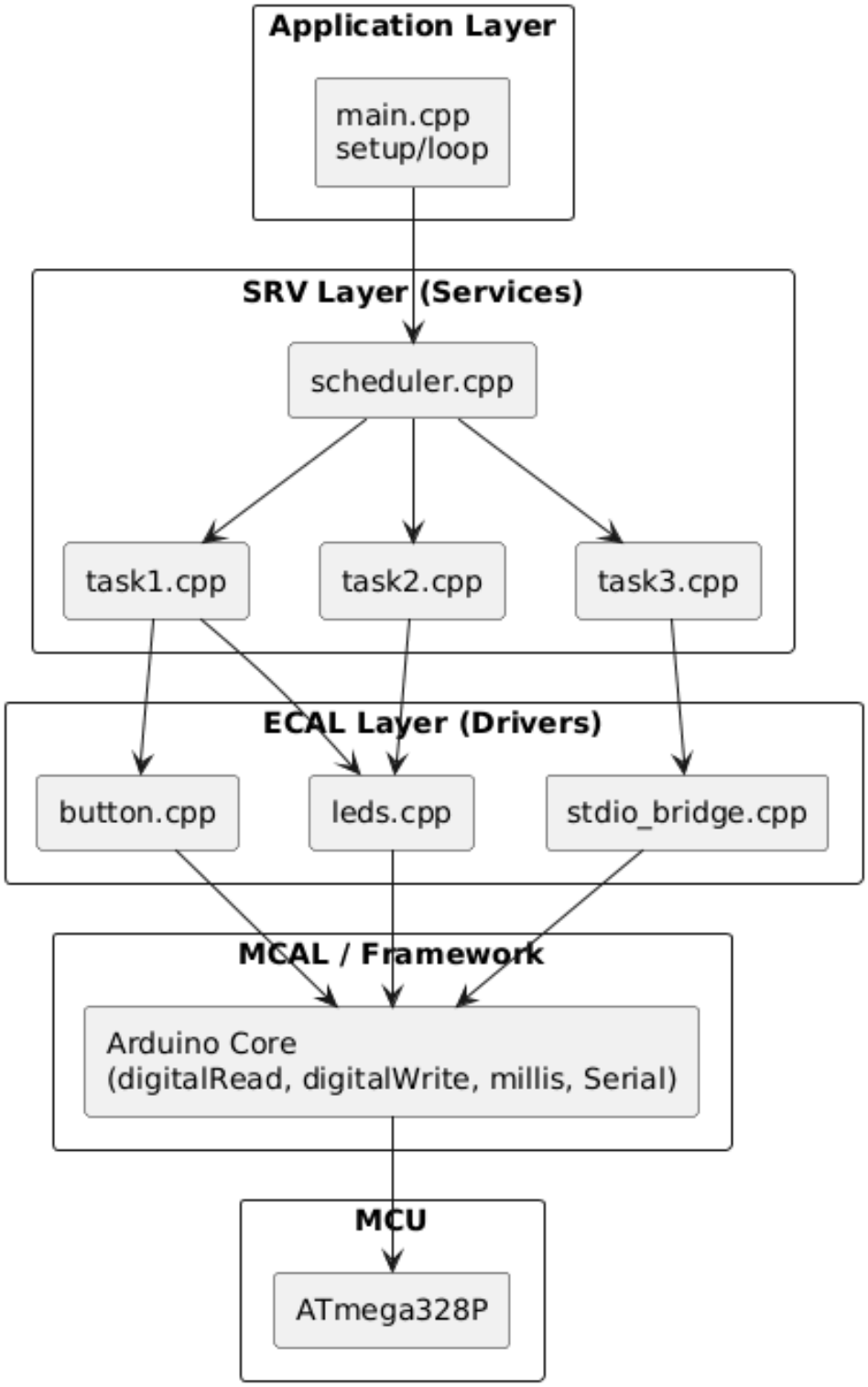


Figure 2: Hardware-software layered interface

This figure highlights the layered separation between application logic, service mod-

ules, driver modules, and the microcontroller abstraction used by the firmware. The diagram explains how responsibilities are distributed across layers so that hardware access remains isolated from task logic. This separation improves maintainability and portability, because low-level pin operations can be changed with minimal impact on higher-level scheduling and application behavior.

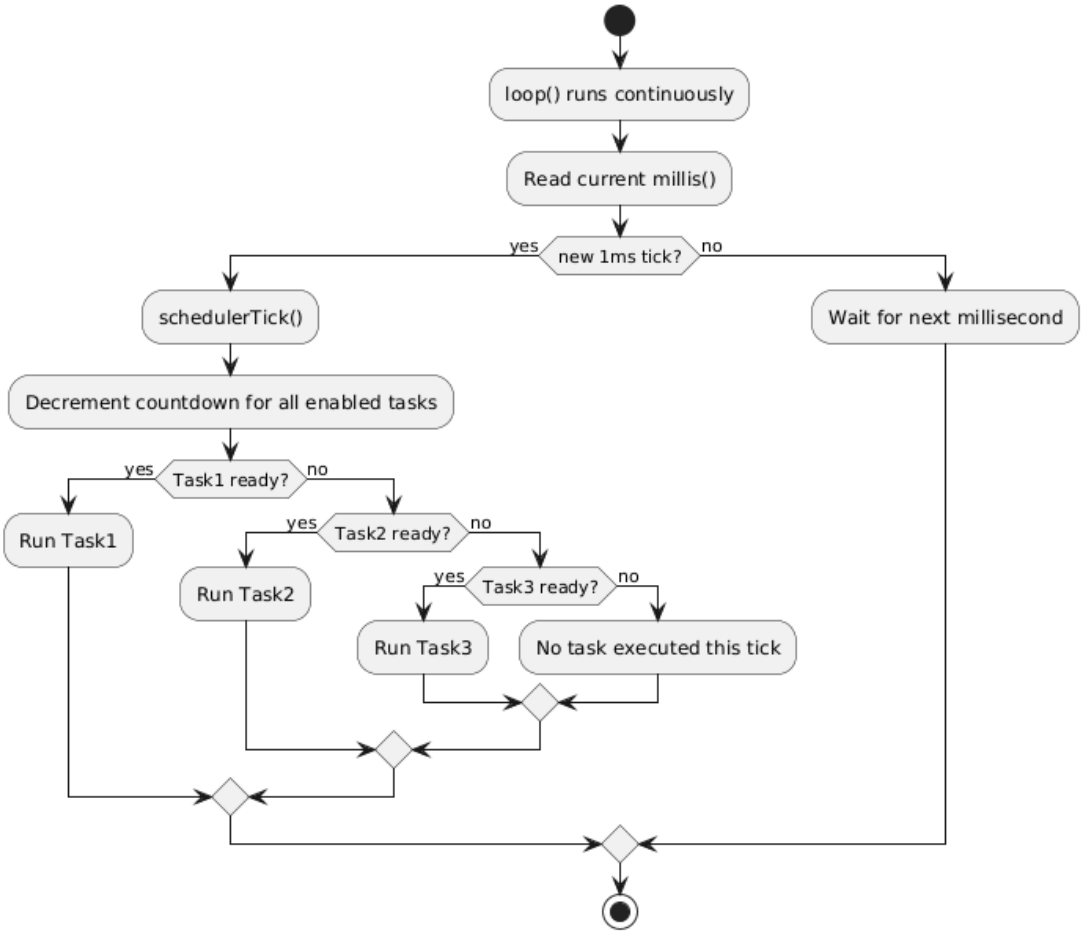


Figure 3: Scheduler flowchart

The scheduler flowchart explains recurrence/offset handling and confirms the non-preemptive policy where at most one ready task is executed per scheduler tick. It details the countdown update step for all configured tasks and the priority-by-order selection policy used in the loop. The chart demonstrates that each activated task runs to completion before the next scheduler decision point, which is consistent with bare-metal cooperative execution.

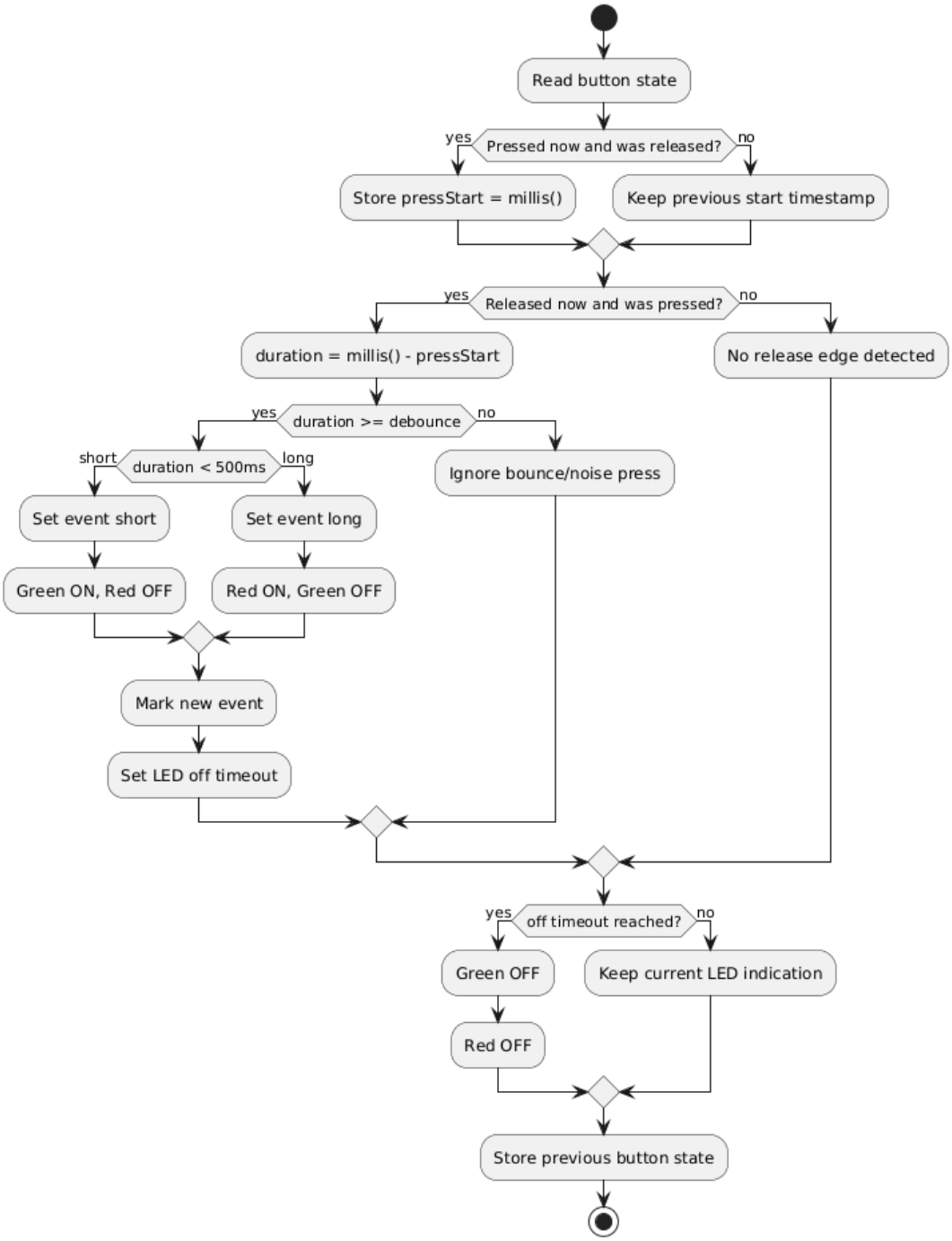


Figure 4: Task 1 flowchart

Task 1 flow shows edge detection, debounce filtering, press-duration measurement, and short/long classification with immediate green/red LED indication. This figure is important because it captures the full decision chain from raw button state transitions to validated press events. It also documents how signal quality is handled through debounce checks and how visual feedback is generated without blocking the scheduler.

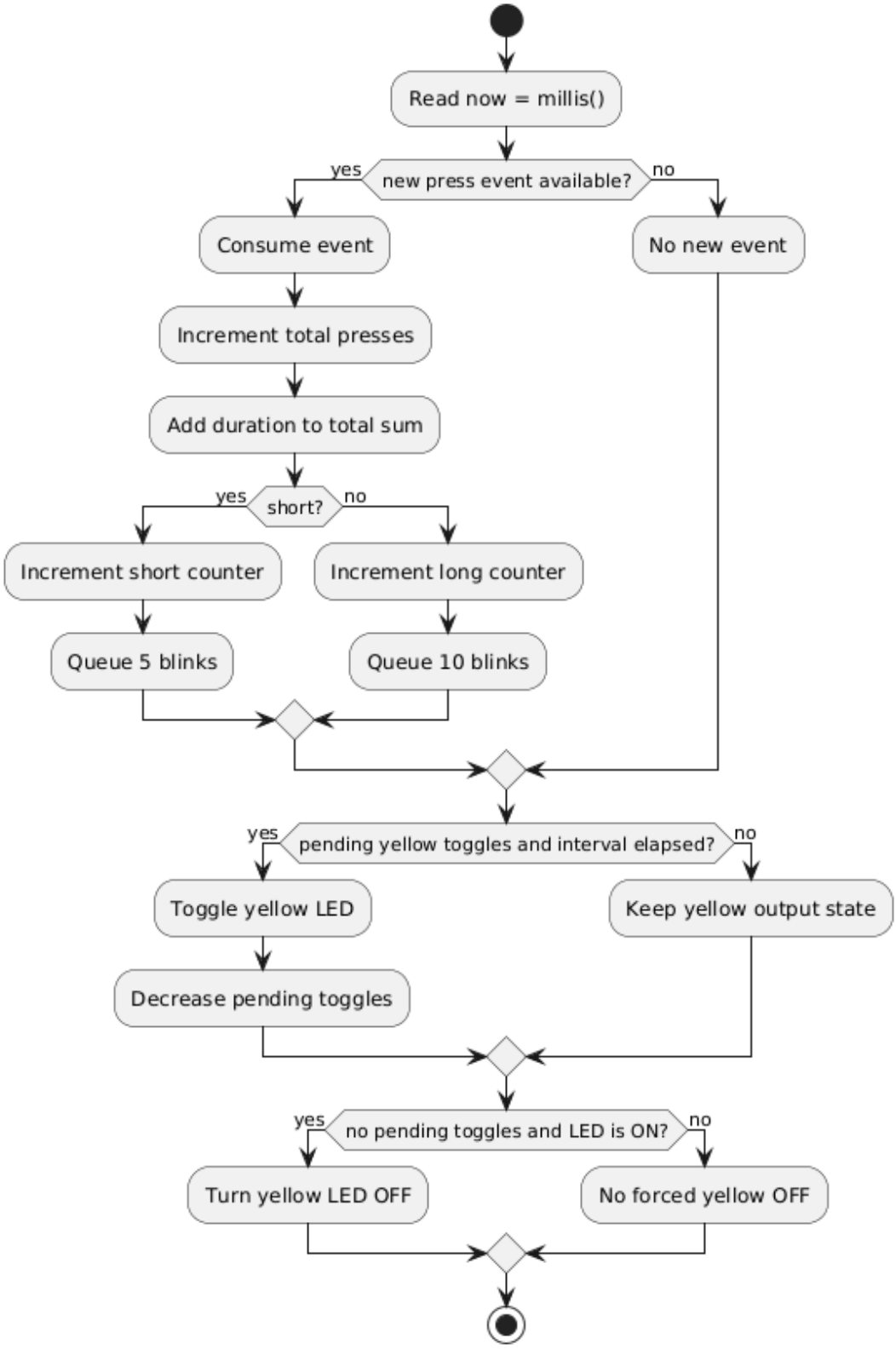


Figure 5: Task 2 flowchart

Task 2 flow details statistics accumulation and the yellow LED blink behavior (5 blinks for short presses and 10 blinks for long presses). The diagram shows both event-driven updates and periodic blink-state progression, which together implement non-

blocking feedback logic. It also clarifies how counters and duration sums are maintained for later reporting by Task 3.

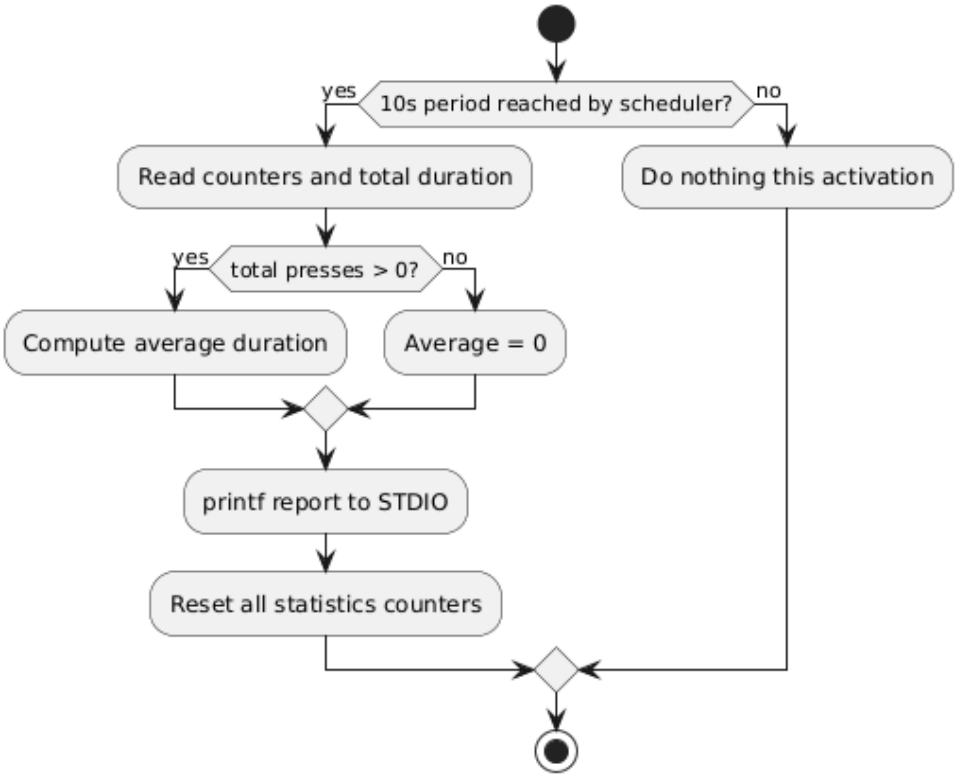


Figure 6: Task 3 flowchart

Task 3 flow describes the 10-second periodic report generation through STDIO and the reset of interval statistics after each report cycle. It highlights the average-duration computation path and the safe handling of the no-data interval case. This confirms that reporting remains deterministic and that each time window starts with clean counters after transmission.

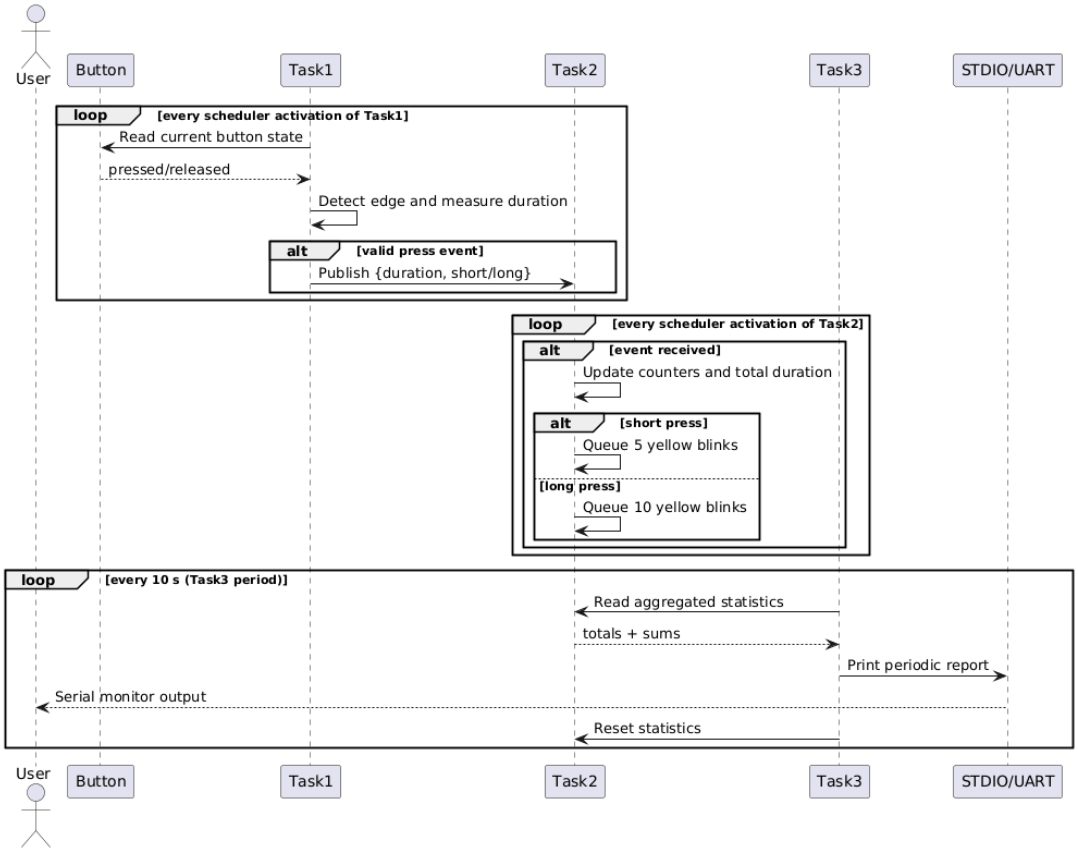
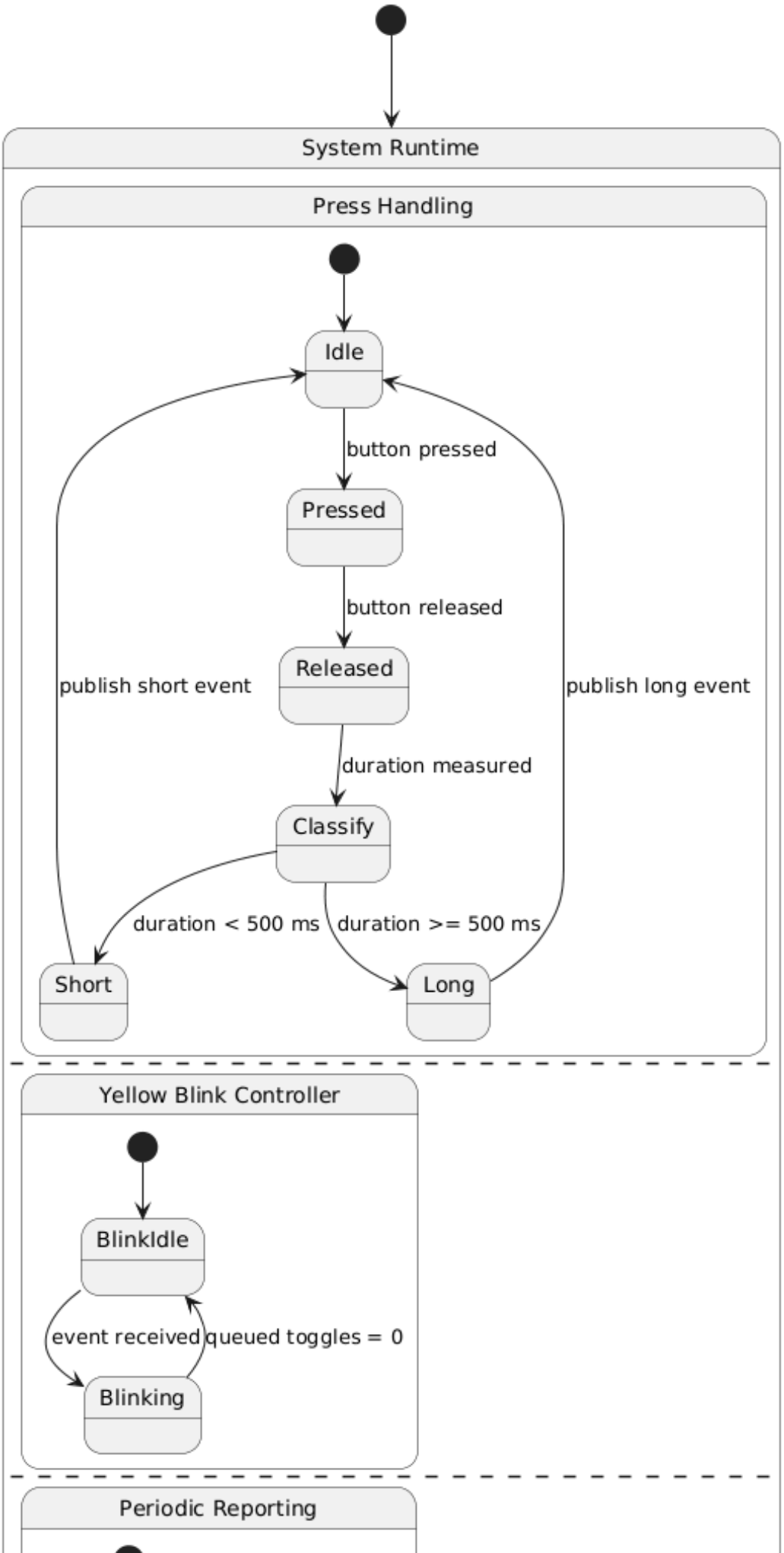


Figure 7: Sequence diagram for inter-task data flow

The sequence diagram illustrates temporal interactions between tasks, including event publication from Task 1 to Task 2 and periodic data extraction by Task 3. It provides a timeline-based view of dependencies, making it easier to verify that data flows in one direction without circular coupling. The sequence also shows where serial output is produced, linking internal state updates to observable external behavior.


System Runtime
Press Handling
Idle
button pressed
Pressed
button released
Released
publish short event
publish long event
duration measured
Classify
duration < 500 ms
duration >= 500 ms
Short
Long
Yellow Blink Controller
BlinkIdle
event received
queued toggles = 0
Blinking
Periodic Reporting

This state-machine view captures the behavioral states of press handling, blink control, and reporting, and clarifies valid transitions during runtime. By grouping related transitions, the FSM explains how concurrent logical behaviors coexist in a cooperative scheduler context. It is especially useful for validating corner cases, such as repeated presses during ongoing blink sequences and transitions back to idle conditions.

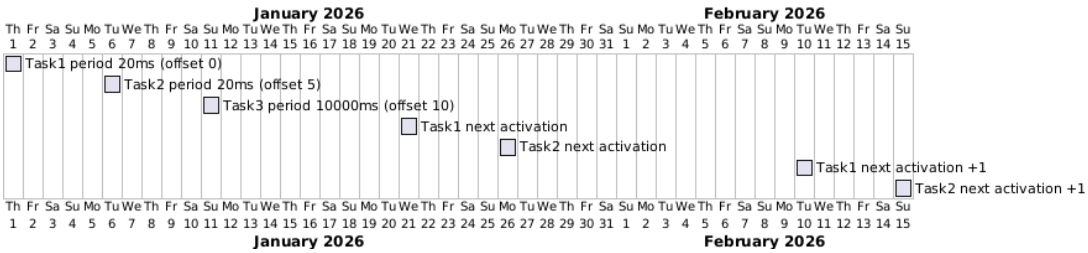


Figure 9: Task timing and offset planning

The timing diagram provides a planning view of task recurrences and offsets, supporting the objective of distributing execution load across ticks. It visually demonstrates how offsets reduce activation overlap and help maintain predictable response timing. This supports the requirement for efficient CPU scheduling and contributes to keeping user-visible latency below the specified limit.

Validation matrix:

Test ID	Scenario	Pass criteria
T-01	Single short press (< 500 ms)	Green LED indication + one event counted
T-02	Single long press (≥ 500 ms)	Red LED indication + one event counted
T-03	Short press feedback	Yellow LED blinks exactly 5 times
T-04	Long press feedback	Yellow LED blinks exactly 10 times
T-05	10-second report	Correct counters + average + reset
T-06	Multiple presses	Aggregation is correct
T-07	Latency check	Visible response below 100 ms

Table 1: Validation scenarios and criteria

Results

The implementation compiled successfully in PlatformIO for Arduino Uno (`pio run` SUCCESS) and was prepared for Wokwi simulation with `diagram.json` and `wokwi.toml`.

The report includes all currently available generated diagram images from the project and dedicated evidence slots for screenshots and optional hardware proof.

Electrical circuit schema

The electrical circuit schema for this lab is the visual Wokwi circuit view generated from the project wiring definition. It must show the Arduino Uno, button on D2 with pull-up behavior, LEDs on D9/D10/D11, and the three 220 Ω current-limiting resistors.

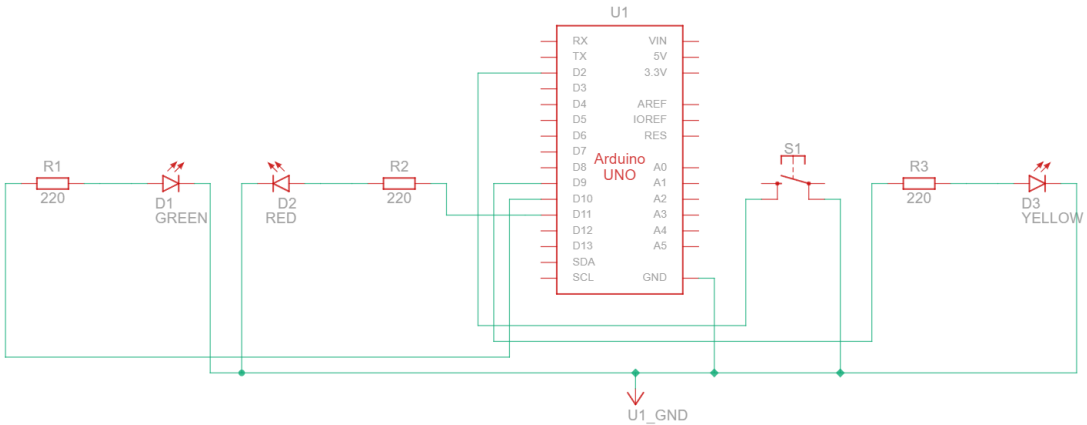


Figure 10: Electrical circuit schema

This figure corresponds to the required electrical circuit schema evidence and should show the exact circuit diagram used for implementation. The final image should clearly expose pin-level connectivity, resistor placement, and component orientation so the setup can be reproduced by another reviewer. Including this evidence links the implemented software directly to a verified hardware topology.

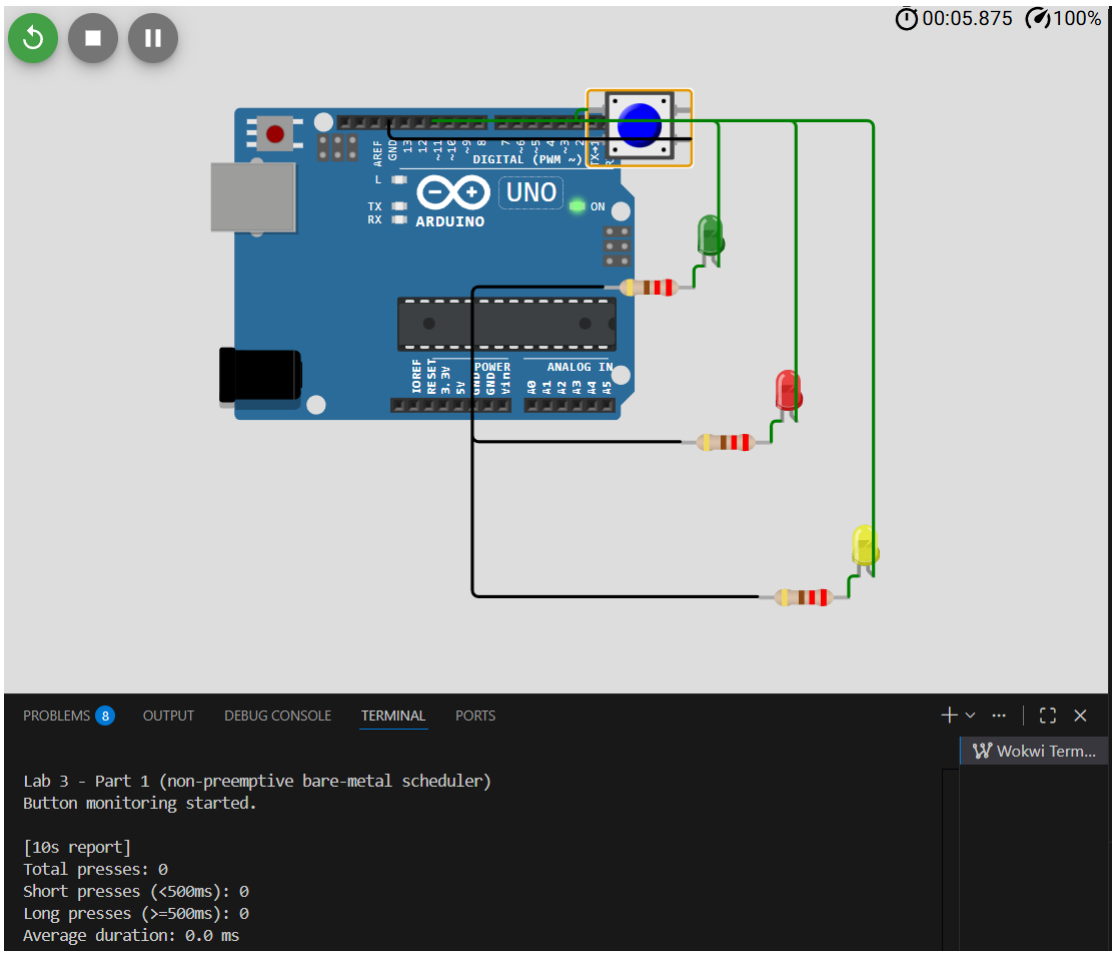


Figure 11: Wokwi running simulation view

This image should prove that the designed schema and firmware execute correctly during simulation. When captured, it should include active LED states or interaction traces that match expected task behavior under button input. This provides runtime confirmation that the architecture and logic are not only syntactically correct but operationally valid.

```
Lab 3 - Part 1 (non-preemptive bare-metal scheduler)
Button monitoring started.

[10s report]
Total presses: 0
Short presses (<500ms): 0
Long presses (>=500ms): 0
Average duration: 0.0 ms

[10s report]
Total presses: 3
Short presses (<500ms): 2
Long presses (>=500ms): 1
Average duration: 498.3 ms

[10s report]
Total presses: 4
Short presses (<500ms): 3
Long presses (>=500ms): 1
Average duration: 358.7 ms

[10s report]
Total presses: 2
Short presses (<500ms): 1
Long presses (>=500ms): 1
Average duration: 819.0 ms

[10s report]
Total presses: 1
Short presses (<500ms): 0
Long presses (>=500ms): 1
Average duration: 4793.0 ms
```

Figure 12: Serial monitor periodic report output

This screenshot documents STDIO functionality and validates the periodic reporting requirement. The figure should show at least one complete 10-second cycle with total,

short, long, and average values in the same report block. It serves as direct evidence for output correctness and helps cross-check the statistics logic against interaction scenarios.

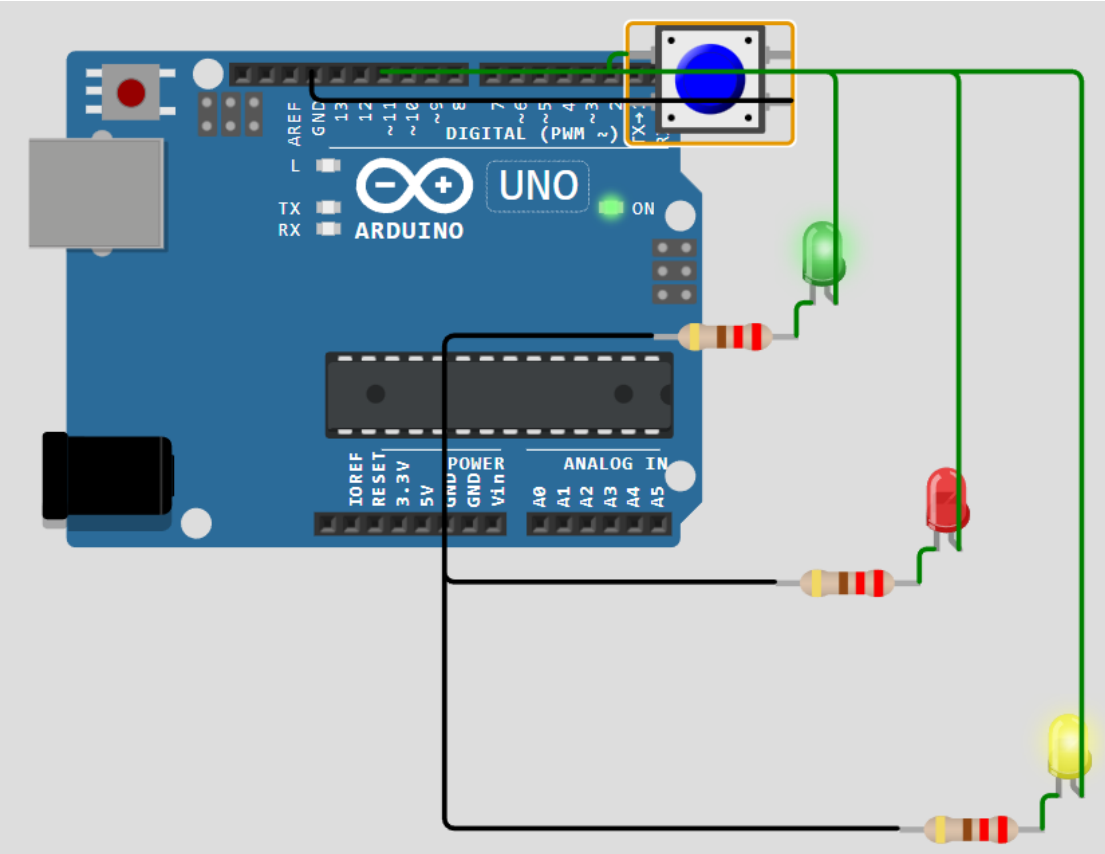


Figure 13: Short press behavior evidence

This evidence should demonstrate correct short-press classification and expected visual feedback behavior. Ideally, the capture should show the green LED indication together with the corresponding yellow blink sequence initiation. This confirms threshold logic and immediate response for short interactions.

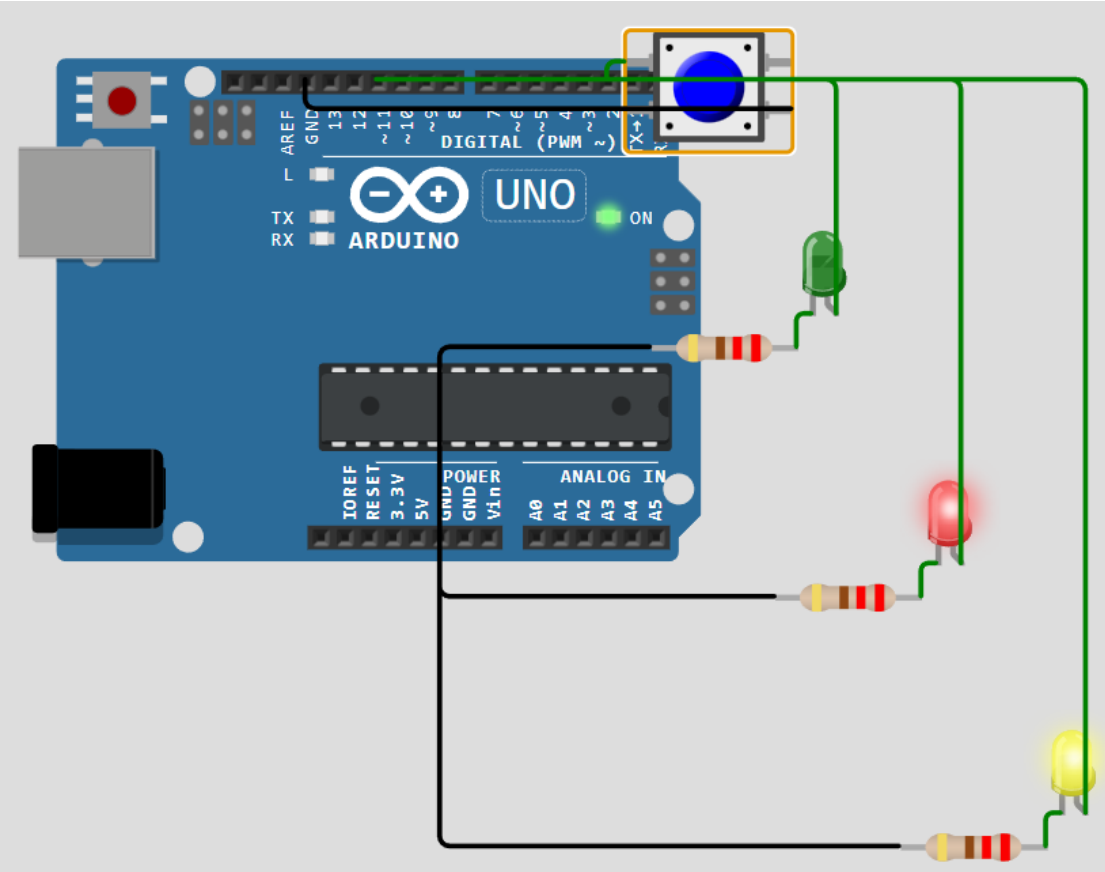


Figure 14: Long press behavior evidence

This evidence should demonstrate correct long-press classification and expected visual feedback behavior. The final image should reflect red LED signaling and the longer yellow blink sequence associated with long presses. This validates both classification boundaries and user feedback consistency.

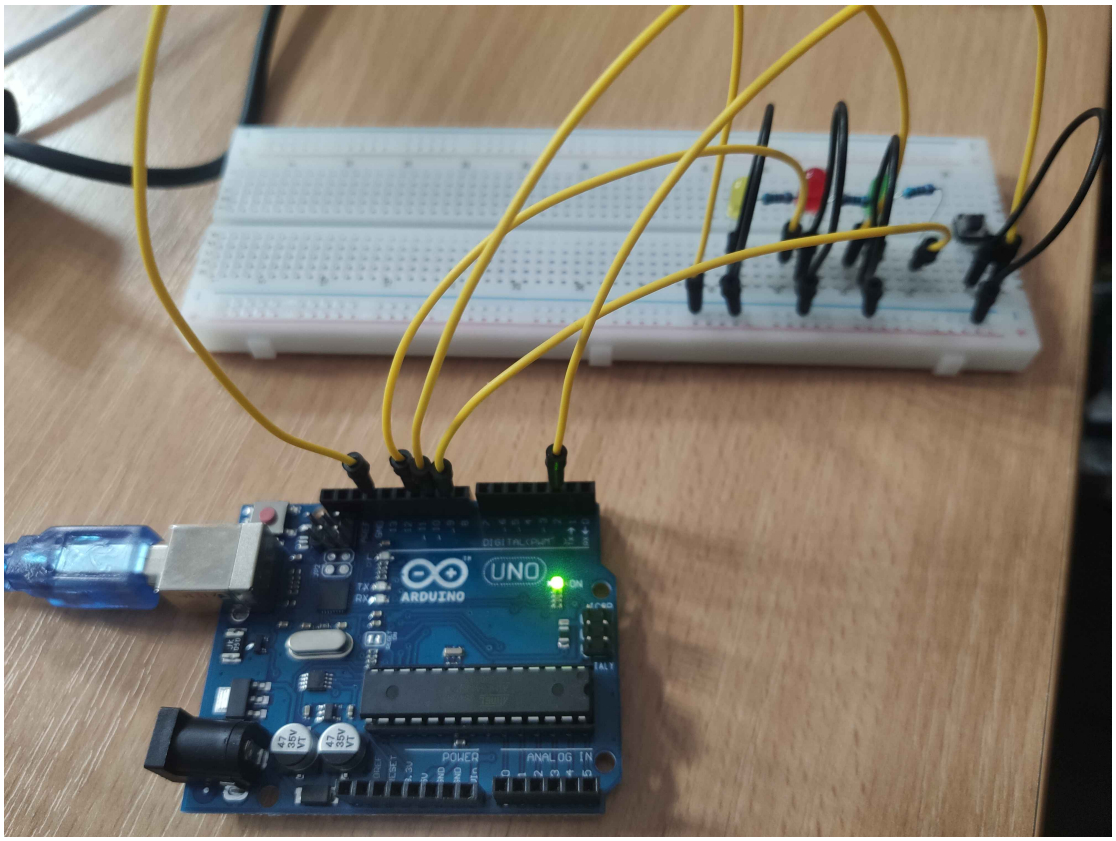


Figure 15: Optional real hardware setup photo

If hardware implementation is available, this photo confirms real-world wiring consistency with the simulation schema; otherwise, simulation-only validation is explicitly documented. A clear top-view image with visible pin mapping is recommended so reviewers can correlate the physical layout with the Wokwi circuit and report diagrams. If no physical setup is used, retaining this placeholder still documents that the project was validated in simulation conditions.

Conclusion

Part 1 of Lab 3 was completed with a non-preemptive bare-metal modular architecture that satisfies required behavior: duration detection, short/long classification, visual signaling, periodic STDIO reporting, and recurrence/offset scheduling. The report structure now follows the original template style while preserving complete technical content and clear evidence slots.

Bibliography

1. Embedded Systems Laboratory Guide, Technical University of Moldova, 2026.

2. ATmega328P Datasheet, Microchip Technology Inc.
3. PlatformIO Documentation. `https://docs.platformio.org/`
4. Wokwi Documentation. `https://docs.wokwi.com/`
5. AVR Libc Documentation. `https://www.nongnu.org/avr-libc/`

Note on AI Tool Usage

During preparation of this report and during code development, AI tools were used for drafting, structuring, and refining content and implementation ideas. The resulting information and code were reviewed, validated, and adjusted according to lab requirements.

Appendices

GitHub repository: `https://github.com/Tirppy/si-course-repo/tree/main/Lab3/code`